4-3 · MinIO–Iceberg 데이터 파이프라인 구축 · 이론 1H

데이터 파이프라인 설계

원본은 그대로 두고, 몇 번 돌려도 같은 결과

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 랜딩 존(raw) → 정제 → 적재로 이어지는 단계를 설계할 수 있다

② 멱등성이 왜 파이프라인의 기본 원칙인지 설명할 수 있다

③ MERGE INTO 가 「있으면 갱신, 없으면 삽입」임을 안다

④ 원천 읽기(s3a)와 Iceberg 쓰기(S3FileIO)의 설정이 다름을 안다

2-5 의 fetch·parse·load 와 구조는 같고 도구만 Spark 와 Iceberg 로 바뀝니다

1. 오늘 만들 것

급식실 배식 라인 — 창고 · 조리대 · 배식대

랜딩 존	정제	적재
식자재 창고 raw 버킷	조리대 Spark ETL	배식대 Iceberg 테이블

2-5 fetch·parse·load · 3-5 하이브리드 파이프라인이 규모를 키운 형태

재료를 곧바로 배식대에 올리지 않는 이유 — 상한 재료를 걸러내고, 같은 재료가 두 번 와도 반찬이 두 배가 되지 않게

2. 랜딩 존

세 층 — 원본은 손대지 않고 위층은 다시 만든다

층	버킷·테이블	무엇이 들어가나	고칠 수 있나
랜딩(raw)	s3://raw/orders/	받은 그대로의 CSV	절대 안 고침
정제(clean)	Iceberg 테이블	타입 맞추고 걸러낸 결과	재실행으로 다시 만듦
집계(mart)	Iceberg 테이블	분석용 요약	재실행으로 다시 만듦

브론즈·실버·골드라고도 부른다 — 이름은 회사마다 달라도 발상은 같다

랜딩 존을 왜 따로 두는가

1. **언제든 여기서 다시 시작**
   원천 파일을 받은 그대로 보관하는 버킷 — 문제가 생기면 여기서 다시 돌린다

2. **원본은 읽기 전용**
   3-5 의 원칙과 같다 — 원본을 지우면 정제 규칙을 바꿔 다시 돌릴 수 없다

3. **원천이 지워도 사본이 남는다**
   원천 시스템이 데이터를 지워도 우리 쪽에는 남아 있다

4. **「어제 파이프라인이 이상했다」**
   그날 받은 원본을 그대로 다시 볼 수 있어야 원인을 찾는다

3. 멱등성

멱등성 — 같은 작업을 몇 번 실행해도 결과가 같다

상황	멱등하지 않으면	멱등하면
중간 실패 후 재시도	앞부분이 두 번 들어간다	아무 문제 없다
스케줄러가 두 번 띄움	데이터가 두 배가 된다	결과가 같다
어제 배치를 다시 돌려 보정	중복이 쌓인다	안심하고 다시 돌린다
원인 조사를 위해 재현	건드리기 무섭다	자유롭게 재현한다

8주차 Airflow 가 실패한 작업을 자동 재시도할 수 있는 것도 각 단계가 멱등하기 때문

2-5 의 UNIQUE + INSERT IGNORE, 3-5 의 「두 번 돌려 equals() 가 True」와 같은 성질 — 오늘은 MERGE INTO 로

MERGE INTO — 있으면 갱신, 없으면 삽입

1. **ON t.키 = s.키**
   무엇이 「같은 것」인가를 정한다 — 오늘은 order_id

2. **WHEN MATCHED THEN UPDATE SET ***
   키가 있으면 덮어쓴다

3. **WHEN NOT MATCHED THEN INSERT ***
   키가 없으면 넣는다 — 조건을 더 붙여 DELETE 도 가능

가장 흔한 실패

원천에 같은 키 여러 행

> [MERGE_CARDINALITY_VIOLATION] ... matched a single row from the target table with multiple rows of the source table

5. 중복 제거

중복 제거는 결정적이어야 한다

나쁜 예 — dropDuplicates

어느 행이 남을지 약속하지 않는다

- raw.dropDuplicates(["order_id"])
- 분산 처리라 실행마다 다른 행이 남는다
- 멱등성이 깨진다

VS

좋은 예 — Window + row_number

「가장 최근 order_date 를 남긴다」를 명시

- partitionBy("order_id").orderBy(desc)
- row_number() 로 번호 → rn = 1 만
- 2-2 의 ROW_NUMBER 그대로

6. 두 갈래 경로

s3a 와 S3FileIO — 같은 MinIO, 설정은 따로

	원천 CSV 읽기	Iceberg 테이블 쓰기
쓰는 것	Hadoop s3a 파일시스템	Iceberg S3FileIO
설정 위치	spark.hadoop.fs.s3a.*	카탈로그 설정(이미 됨)
주소 지정	s3a://raw/orders/	카탈로그가 알아서
자격증명	직접 넘겨야 함	컨테이너 환경변수로
부품	hadoop-aws(이미지에 없음)	iceberg-aws-bundle(있음)

카탈로그 설정이 있어도 s3a 는 따로 — 그래서 --packages 하나(부품)와 --conf 네 개(MinIO 주소·계정)를 붙인다

정리

이 1시간에서 꼭 가져갈 것

1. 원본은 랜딩 존에 그대로 두고, 아래층에서 위층을 다시 만든다
2. 멱등성은 「몇 번 돌려도 같은 결과」 — 재시도를 안전하게 만든다
3. MERGE INTO 는 있으면 갱신, 없으면 삽입으로 멱등 적재를 만든다
4. MERGE 전에 키 중복을 하나로 줄이되, 규칙을 명시해 결정적으로
5. 원천 읽기는 s3a, Iceberg 쓰기는 S3FileIO — 설정이 따로다

4-3 · 이론 1H 마무리

다음은 원천 데이터 적재 실습 (1H)

랜딩 존에 새 배치 파일을 올리고, ETL 을 짜기 전에 Spark 가 그것을 읽을 수 있는지부터 확인합니다

- ✓ 4-2 컨테이너 4개와 orders 4행 확인
- ✓ orders_2026-08-02.csv 를 raw 버킷에
- ✓ spark-shell 에 --packages 와 --conf 네 개
- ✓ 겹치는 order_id 2 를 눈으로 확인

준비할 것

4-2 의 ~/lakehouse 폴더 · 도커 실행 상태